

## TP N°6: TDD - Test Driven Development



Guía de pruebas a desarrollar y ejecutar para la implementación de la funcionalidad de “Inscribirse a actividad”

### GRUPO 8

#### DOCENTES:

- Ing. Laura Covaro
- Ing. Cecilia Massano
- Ing. Constanza Garnero
- Ing. Judith Meles

#### INTEGRANTES:

- 87834 - Barrionuevo Natalia
- 80738 - Benjamin, Ian Nicolas
- 81699 - Benedetti, Angelo
- 84066 - Brito, Serena
- 71471 - Enrique, Walter Ignacio
- 98965 - Fernandez, Santiago
- 81705 - García, Valentina
- 85296 - Gomez Alameda, Romina Abigail
- 82397 - Sosa, Diego
- 82127 - Tarquinio, Angel Valentín

## Casos de prueba

### CP-01 — Registro exitoso con actividad que requiere vestimenta (pasa)

#### Precondiciones

- Existe la actividad **Tirolesa** con horario **15:00 disponible**.
- Hay al menos **1 cupo libre** en el horario **15:00**.
- La actividad **requiere talla** de vestimenta.
- **Hora actual: 08:30** (anterior al horario de inicio).

#### Pasos

1. El visitante selecciona la actividad **Tirolesa**.
2. El visitante selecciona el horario **15:00**.
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa los datos: **Nombre = Juan Pérez, DNI = 12345678, Edad = 25, Talla = M**.
5. El visitante **marca "Acepto Términos y Condiciones"**.
6. El visitante **confirma la inscripción**.

#### Resultados esperados

7. El sistema muestra un **mensaje de confirmación** "Registro exitoso".
8. Se crea el registro de inscripción asociado a la actividad y horario.
9. Los cupos del horario disminuyen en 1.

```
1 def test_1_successful_registration_with_clothing_required(self):
2     """Prueba 1: Registro exitoso con actividad que requiere vestimenta"""
3     with self.app.app_context():
4         # Crear actividad Tirolesa para este test
5         tirolesa = Activity(
6             name="Tirolesa",
7             capacity=10,
8             schedules=["15:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirolesa)
13        db.session.commit()
14
15        visitor_data = {
16            'participants': [{
17                'name': 'Juan Pérez',
18                'dni': '12345678',
19                'age': 25,
20                'clothing_size': 'M'
21            }],
22            'terms_accepted': True,
23            'participants_count': 1,
24            'current_time': '08:30'
25        }
26
27        result = ActivityService.register_visitor(
28            activity_id=tirolesa.id,
29            visitor_data=visitor_data,
30            schedule='15:00'
31        )
32
33        assert result['success'] == True
34        assert 'Registro exitoso' in result['message']
35
36        # Verificar que se creó el registro
37        registration = Registration.query.filter_by(
38            activity_id=tirolesa.id
39        ).first()
40        assert registration is not None
```

## CP-02 — Sin cupos disponibles (falla)

### Precondiciones

- Actividad **Tirolesa**, horario **15:00** tiene **12 cupos**, todos **ocupados**.
- Hora actual: **08:30** (anterior al horario de inicio).

### Pasos

1. El visitante selecciona la actividad **Tirolesa**.
2. El visitante selecciona el horario **15:00**.
3. El visitante ingresa **Cantidad = 1**.
4. El visitante completa sus datos: **Nombre = Marco Polo**, **DNI = 49999999**, **Edad 20**, **Talla L**.
5. El visitante **acepta** Términos y Condiciones.
6. El visitante **Confirma la inscripción**.

### Resultados esperados

7. El sistema **rechaza** la inscripción y muestra el mensaje **“No hay cupos disponibles”**.
8. **No** se crea registro ni se modifica la disponibilidad.

```

1  def test_2_no_available_slots_fails(self):
2      """Prueba 2: Sin cupos disponibles - debe fallar"""
3      with self.app.app_context():
4          # Crear actividad Tirolesa y Llenar todos los cupos
5          tirolesa = Activity(
6              name="Tirolesa",
7              capacity=12,
8              schedules=["15:00"],
9              requirements={"nivel": "todos"},
10             requires_clothing=True
11         )
12         db.session.add(tirolesa)
13         db.session.commit()
14
15         # Llenar todos los cupos
16         for i in range(12):
17             visitor = Visitor(
18                 name=f'Visitante {i+1}',
19                 dni=f'1234567{i}',
20                 age=25,
21                 terms_accepted=True
22             )
23             db.session.add(visitor)
24             db.session.flush()
25
26             registration = Registration(
27                 activity_id=tirolesa.id,
28                 visitor_id=visitor.id,
29                 schedule='15:00'
30             )
31             db.session.add(registration)
32
33         db.session.commit()
34
35         # Intentar registrar un visitante más
36         visitor_data = {
37             'participants': [{
38                 'name': 'Marco Polo',
39                 'dni': '49999999',
40                 'age': 20,
41                 'clothing_size': 'L'
42             }],
43             'terms_accepted': True,
44             'participants_count': 1,
45             'current_time': '08:30'
46         }
47
48         result = ActivityService.register_visitor(
49             activity_id=tirolesa.id,
50             visitor_data=visitor_data,
51             schedule='15:00'
52         )
53
54         assert result['success'] == False
55         assert 'No hay cupos disponibles' in result['error']

```

### CP-03 — Actividad sin vestimenta requerida (pasa)

#### Precondiciones

- La actividad **Jardinería** se encuentra cargada y con cupos libres.
- Hora actual: **08:30** (anterior al horario de inicio).

#### Pasos

1. El visitante selecciona **Jardinería**.
2. El visitante selecciona el horario **16:00**.
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa sus datos: **Nombre = María García, DNI = 34567890, Edad = 30, Talla = vacío**.
5. El visitante **acepta** Términos y Condiciones.
6. El visitante **confirma la inscripción**.

#### Resultados esperados

7. El sistema **muestra un mensaje “Registro exitoso”**.
9. El sistema **disminuye los cupos** del horario en 1.

```

1  def test_3_clothing_size_not_required_passes(self):
2      """Prueba 3: Actividad sin vestimenta requerida - debe pasar sin talla"""
3      with self.app.app_context():
4          # Crear actividad Jardinería
5          jardineria = Activity(
6              name="Jardinería",
7              capacity=12,
8              schedules=["16:00"],
9              requirements={"nivel": "todos"},
10             requires_clothing=False
11         )
12         db.session.add(jardineria)
13         db.session.commit()
14
15         visitor_data = {
16             'participants': [{
17                 'name': 'María García',
18                 'dni': '34567890',
19                 'age': 30,
20                 'clothing_size': None # Sin talla
21             }],
22             'terms_accepted': True,
23             'participants_count': 1,
24             'current_time': '08:30'
25         }
26
27         result = ActivityService.register_visitor(
28             activity_id=jardineria.id,
29             visitor_data=visitor_data,
30             schedule='16:00'
31         )
32
33         assert result['success'] == True
34         assert 'Registro exitoso' in result['message']

```

## CP-04 — Horario que ya pasó (falla)

### Precondiciones

- La actividad **Tirolesa** se encuentra cargada y posee el horario **14:00**.
- Hora actual: **16:30**.

### Pasos

1. El visitante selecciona la actividad **Tirolesa**.
2. El visitante selecciona el horario **14:00**.
3. El visitante completa los datos: **Nombre = Carlos López, DNI = 31223344, Edad = 35, Talla = L**.
4. El visitante **acepta** Términos y Condiciones.
5. El visitante **confirma la inscripción**.

### Resultados esperados

6. El sistema muestra un **mensaje de error “El horario ya pasó”**
7. El sistema no crea registro ni se alteran cupos.



```
1 def test_4_invalid_schedule_fails(self):
2     """Prueba 4: Horario no disponible - debe fallar"""
3     with self.app.app_context():
4         # Crear actividad Tirolesa
5         tirolesa = Activity(
6             name="Tirolesa",
7             capacity=10,
8             schedules=["14:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirolesa)
13        db.session.commit()
14
15        visitor_data = {
16            'participants': [{
17                'name': 'Carlos López',
18                'dni': '31223344',
19                'age': 35,
20                'clothing_size': 'L'
21            }],
22            'terms_accepted': True,
23            'participants_count': 1,
24            'current_time': '16:30' # Hora posterior al horario
25        }
26
27        result = ActivityService.register_visitor(
28            activity_id=tirolesa.id,
29            visitor_data=visitor_data,
30            schedule='14:00' # Horario que ya pasó
31        )
32
33        assert result['success'] == False
34        assert 'ya pasó' in result['error']
```

## CP-05 — Términos y Condiciones no aceptados (falla)

### Precondiciones

- Existe la actividad **Tirolesa** con horario **15:00** y **cupos libres**.
- Hora actual: **08:30**.

### Pasos

1. El visitante selecciona la actividad **Tirolesa**.
2. El visitante selecciona el horario de las **15:00**.
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa datos: **Nombre = Ana Martínez**, **DNI = 55667788**, **Edad 18**, **Talla S**.
5. El visitante **no marca** la casilla de aceptación de Términos y Condiciones.
6. El visitante **confirma la inscripción**

### Resultados esperados

7. El sistema **rechaza** la inscripción con el mensaje “**Debe aceptar los Términos y Condiciones**”.
8. **No** se crea registro ni se alteran cupos.

```
1 def test_5_terms_not_accepted_fails(self):
2     """Prueba 5: Términos no aceptados - debe fallar"""
3     with self.app.app_context():
4         # Crear actividad Tirolesa
5         tirolesa = Activity(
6             name="Tirolesa",
7             capacity=10,
8             schedules=["15:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirolesa)
13        db.session.commit()
14
15        visitor_data = {
16            'participants': [{
17                'name': 'Ana Martínez',
18                'dni': '55667788',
19                'age': 18,
20                'clothing_size': 'S'
21            }],
22            'terms_accepted': False, # No acepta términos
23            'participants_count': 1,
24            'current_time': '08:30'
25        }
26
27        result = ActivityService.register_visitor(
28            activity_id=tirolesa.id,
29            visitor_data=visitor_data,
30            schedule='15:00'
31        )
32
33        assert result['success'] == False
34        assert 'aceptar los términos y condiciones' in result['error']
```

## CP-06 — Falta talla requerida (falla)

### Precondiciones

- Existe la actividad **Tirolesa** con horario **15:00** y **cupos libres**.
- La actividad **requiere talla de vestimenta**.
- Hora actual: **08:30**.

### Pasos

1. El visitante selecciona la actividad **Tirolesa**.
2. El visitante selecciona el horario de las **15:00**.
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa datos: **Nombre = Pedro Rodríguez, DNI = 49887766, Edad 20, Talla = vacío**.
5. El visitante **acepta** Términos y Condiciones.
6. El visitante **confirma la inscripción**.

### Resultados esperados

7. El sistema **rechaza** la inscripción con el mensaje "La actividad requiere especificar talla".
8. **No** se crea registro ni se alteran cupos.

```
1 def test_6_required_clothing_size_missing_fails(self):
2     """Prueba 6: Talla requerida faltante - debe fallar"""
3     with self.app.app_context():
4         # Crear actividad Tirolesa
5         tirolesa = Activity(
6             name="Tirolesa",
7             capacity=10,
8             schedules=["15:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirolesa)
13        db.session.commit()
14
15        visitor_data = {
16            'participants': [{
17                'name': 'Pedro Rodríguez',
18                'dni': '49887766',
19                'age': 20,
20                'clothing_size': None # Sin talla cuando es requerida
21            }],
22            'terms_accepted': True,
23            'participants_count': 1,
24            'current_time': '08:30'
25        }
26
27        result = ActivityService.register_visitor(
28            activity_id=tirolesa.id,
29            visitor_data=visitor_data,
30            schedule='15:00'
31        )
32
33        assert result['success'] == False
34        assert 'requiere especificar talla' in result['error']
```

## CP-07 — Inscripción múltiple exitosa (pasa)

### Precondiciones

- Existe la actividad **Tirolesa** con horario **11:00** y **al menos 2 cupos libres**.
- La actividad **requiere talla de vestimenta**.
- Hora actual: **08:30**.

### Pasos

1. El visitante selecciona **Tirolesa**.
2. El visitante selecciona el horario de las **11:00**.
3. El visitante ingresa **Cantidad de participantes = 2**.
4. El visitante completa los datos:
  - **Nombre = Macarena Pasos, DNI = 41111111, Edad = 20, Talla M.**
  - **Nombre = Camila Pasos, DNI = 42222222, Edad = 19, Talla L.**
5. El visitante **acepta** Términos y Condiciones.
6. El visitante **confirma la inscripción**.

### Resultados esperados

7. El sistema muestra un **mensaje “Registro exitoso”**.
8. Se **crean 2 registros** asociados al horario **11:00**.
9. El sistema **reduce los cupos** del horario en **2**.

```

1  def test_7_multiple_participants_successful(self):
2      """Prueba adicional: Múltiples participantes exitoso"""
3      with self.app.app_context():
4          # Crear actividad Tirollesa
5          tirolesa = Activity(
6              name="Tirollesa",
7              capacity=10,
8              schedules=["11:00"],
9              requirements={"nivel": "todos"},
10             requires_clothing=True
11         )
12         db.session.add(tirolesa)
13         db.session.commit()
14
15         visitor_data = {
16             'participants': [
17                 {
18                     'name': 'Macarena Pasos',
19                     'dni': '41111111',
20                     'age': 20,
21                     'clothing_size': 'M'
22                 },
23                 {
24                     'name': 'Camila Pasos',
25                     'dni': '42222222',
26                     'age': 19,
27                     'clothing_size': 'L'
28                 }
29             ],
30             'terms_accepted': True,
31             'participants_count': 2,
32             'current_time': '08:30'
33         }
34
35         result = ActivityService.register_visitor(
36             activity_id=tirolesa.id,
37             visitor_data=visitor_data,
38             schedule='11:00'
39         )
40
41         assert result['success'] == True
42         assert 'Registro exitoso' in result['message']
43
44         # Verificar que se crearon ambos registros
45         registrations = Registration.query.filter_by(
46             activity_id=tirolesa.id
47         ).all()
48         assert len(registrations) == 2

```

## CP-08 — Múltiples participantes exceden capacidad (falla)

### Precondiciones

- Se encuentra cargada la actividad **Tirolesa** con horario **15:00** y **12 cupos totales**.
- Hay 11 cupos **ya ocupados** (queda solo 1 disponible).
- Hora actual: **08:30**.

### Pasos

1. El visitante selecciona **Tirolesa**.
2. El visitante selecciona el horario de las **15:00**.
3. El visitante ingresa **Cantidad de participantes = 2**.
4. El visitante completa los datos:
  - **Nombre = Liliana Mirasol, DNI = 33333333, Edad = 30, Talla M.**
  - **Nombre = Narella Villa, DNI = 44444444, Edad = 21, Talla L.**
5. El visitante **acepta** Términos y Condiciones.
6. El visitante **confirma la inscripción**

### Resultados esperados

7. El sistema **rechaza** la inscripción y muestra el mensaje **“No hay cupos disponibles”**.
8. **No** se crean registros ni se modifican los cupos existentes.



```
1 def test_8_multiple_participants_exceeds_capacity_fails(self):
2     """Prueba adicional: Múltiples participantes exceden capacidad - debe fallar"""
3     with self.app.app_context():
4         # Crear actividad Tirollesa
5         tirollesa = Activity(
6             name="Tirollesa",
7             capacity=12,
8             schedules=["15:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirollesa)
13        db.session.commit()
14
15        # Llenar 11 de Los 12 cupos disponibles
16        for i in range(11):
17            visitor = Visitor(
18                name=f'Visitante {i+1}',
19                dni=f'1234567{i}',
20                age=25,
21                terms_accepted=True
22            )
23            db.session.add(visitor)
24            db.session.flush()
25
26            registration = Registration(
27                activity_id=tirollesa.id,
28                visitor_id=visitor.id,
29                schedule='15:00'
30            )
31            db.session.add(registration)
32
33        db.session.commit()
34
35        # Intentar registrar 2 personas más (total 2, pero solo hay 1 cupo)
36        visitor_data = {
37            'participants': [
38                {
39                    'name': 'Liliana Mirasol',
40                    'dni': '33333333',
41                    'age': 30,
42                    'clothing_size': 'M'
43                },
44                {
45                    'name': 'Narella Villa',
46                    'dni': '44444444',
47                    'age': 21,
48                    'clothing_size': 'L'
49                }
50            ],
51            'terms_accepted': True,
52            'participants_count': 2,
53            'current_time': '08:30'
54        }
55
56        result = ActivityService.register_visitor(
57            activity_id=tirollesa.id,
58            visitor_data=visitor_data,
59            schedule='15:00'
60        )
61
62        assert result['success'] == False
63        assert 'No hay cupos disponibles' in result['error']
```

## CP-09 — Sin horario seleccionado (falla)

### Precondiciones

- Existe la actividad **Tirolesa** con horario **15:00** publicado.
- Hay **cupos disponibles** para el horario **15:00**.
- La actividad **requiere** talla de vestimenta.
- Hora actual: **08:30**.

### Pasos

1. El visitante selecciona la actividad **Tirolesa**.
2. El visitante **no** selecciona un horario (campo horario vacío).
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa los datos: **Nombre = María González, DNI = 12345678, Edad = 25, Talla = M**.
5. El visitante marca **“Acepto Términos y Condiciones”**.
6. El visitante **confirma** la inscripción.

### Resultados esperados

7. El sistema **rechaza** la operación y muestra el mensaje **“Se debe seleccionar un horario”**.
8. **No** se crea el registro de inscripción.
9. **No** se alteran los cupos del horario.

```
1 def test_9_no_schedule_selected_fails(self):
2     """Prueba 9: Sin horario seleccionado - debe fallar"""
3     with self.app.app_context():
4         # Crear actividad Tirollesa
5         tirollesa = Activity(
6             name="Tirollesa",
7             capacity=10,
8             schedules=["15:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirollesa)
13        db.session.commit()
14
15        visitor_data = {
16            'participants': [{
17                'name': 'María González',
18                'dni': '12345678',
19                'age': 25,
20                'clothing_size': 'M'
21            }],
22            'terms_accepted': True,
23            'participants_count': 1,
24            'current_time': '08:30'
25        }
26
27        result = ActivityService.register_visitor(
28            activity_id=tirollesa.id,
29            visitor_data=visitor_data,
30            schedule='' # Horario vacío
31        )
32
33        assert result['success'] == False
34        assert 'Horario no disponible' in result['error']
```

## CP-10 — Horario fuera del rango válido / no publicado (falla)

### Precondiciones

- Existe la actividad **Tirolesa** con horarios **desde las 09:00 hasta las 17:30** incluido publicados (no existen **08:30** ni **18:30**).
- La actividad **requiere** talla de vestimenta.
- Hora actual: **8:00**.

### Pasos

1. El visitante selecciona **Tirolesa**.
2. El visitante intenta inscribirse en **08:30** (antes de 09:0).
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa datos: **Nombre = Juan Pérez, DNI = 11111111, Edad = 25, Talla = M**.
5. El visitante acepta T&C.
6. El visitante confirma la inscripción.

### Resultados esperados

7. El sistema **rechaza** la inscripción.
8. **No** se crea registro de inscripción.
9. **No** se modifican los cupos de la actividad.

```
1 def test_10_invalid_time_range_fails(self):
2     """Prueba 10: Horario fuera del rango válido (antes de las 9:00) - debe fallar"""
3     with self.app.app_context():
4         # Crear actividad Tirollesa
5         tirollesa = Activity(
6             name="Tirollesa",
7             capacity=10,
8             schedules=["15:00"],
9             requirements={"nivel": "todos"},
10            requires_clothing=True
11        )
12        db.session.add(tirollesa)
13        db.session.commit()
14
15        # Test con horario antes de las 9:00
16        visitor_data_early = {
17            'participants': [{
18                'name': 'Juan Pérez',
19                'dni': '11111111',
20                'age': 25,
21                'clothing_size': 'M'
22            }],
23            'terms_accepted': True,
24            'participants_count': 1,
25            'current_time': '08:00'
26        }
27
28        result_early = ActivityService.register_visitor(
29            activity_id=tirollesa.id,
30            visitor_data=visitor_data_early,
31            schedule='08:30' # Horario antes de las 9:00
32        )
33
34        assert result_early['success'] == False
35        assert 'Horario no disponible' in result_early['error']
```

## CP-11 — DNI con formato inválido (falla)

### Precondiciones

- Existe la actividad **Tirolesa** con horario **15:00** disponible y **cupos libres**.
- La actividad **requiere** talla de vestimenta.
- Hora actual: **08:30**

### Pasos

1. El visitante selecciona **Tirolesa**.
2. El visitante selecciona el horario de las **15:00**.
3. El visitante ingresa **Cantidad de participantes = 1**.
4. El visitante completa los datos: **Nombre = Carlos López**, **DNI = ABC12345** (no numérico), **Edad = 25**, **Talla = M**.
5. El visitante acepta T&C.
6. El visitante **confirma** la inscripción.

### Resultados esperados

7. El sistema rechaza la inscripción por validación de datos y muestra un **mensaje de error** en datos del participante (DNI inválido).
8. **No** se crea el registro de inscripción.
9. **No** se alteran los cupos del horario.

## Conversación con el PO:

Hola Profe, somos del grupo 8 de la materia ISW del curso 4K3, y tenemos algunas preguntas para hacerle sobre la implementación del producto y detalles de los requerimientos.

1) ¿El horario diario cambia o lo tomamos como fijo para todos los días?

**El rango horario es siempre el mismo, de 9 a 18 horas. Cada actividad dura aproximadamente 30 minutos.**

2) ¿los visitantes se inscriben con anticipación o el mismo día de la actividad?

**Las inscripciones se hacen el mismo día, pero para un horario futuro. Por ejemplo, se pueden anotar hasta un minuto antes del inicio de la actividad.**

3) ¿Existe alguna paleta de colores que debamos usar como referencia?

**Si. <https://coolors.co/134611-3e8914-3da35d-96e072-e8fccf>**

4) ¿Hay alguna actividad que requiera indicar la talla de vestimenta?

**Sí, solo Tirollesa y Palestra piden la talla, por cuestiones de seguridad y equipamiento.**

5) ¿Cuál es el máximo de personas por inscripción en cada actividad?

**Palestra y Jardinería: hasta 12 personas por turno.**

**Safari: máximo 8 personas por turno.**

**Tirollesa: hasta 10 personas por turno.**

**Además, una sola persona puede inscribirse y ocupar todo el cupo si así lo desea.**

6) ¿Hay algún límite de edad para participar en las actividades?

**Sí, hay edad mínima en algunas:**

**Palestra: mínimo 12 años.**

**Tirollesa: mínimo 8 años.**

7) ¿Las inscripciones son solo para el mismo día? ¿No hay posibilidad de anotarse para días futuros?

**Exactamente, solo se puede inscribir el mismo día. No hay reservas anticipadas para otras fechas.**

8) ¿Hay algún día en que el parque no abra o no se realicen actividades?

**No, el parque abre todos los días y siempre hay actividades disponibles.**

Muchas gracias por su tiempo. Saludos profe.